

4_202: Introduction to Django-REST framework

```
# pip install djangorestframework==3.11.1

settings.py
INSTALLED_APPS = [
...
    'rest_framework'
]

REST_FRAMEWORK = {
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.AllowAny',
    ]
}
```

4_205: Implementing GET Endpoints

```
serliarisers.py
from rest_framework import serializers
from .models import *

class GeneSerializer(serializers.Serializer):
    gene_id = serializers.CharField(required=True,
allow_blank=False, max_length=256)
    entity = serializers.CharField((required=True,
allow_blank=False, max_length=256)
    start = serializers.IntegerField()

serliarisers.py
class GeneSerializer(serializers.ModelSerializer):
    class Meta:
        model = Gene
        fields = ['gene_id', 'entity', 'start', 'stop', 'sense',
            'start_codon']
```

```
api.py
from django.http import JsonResponse
from django.views.decorators.csrf import csrf_exempt
from rest_framework.parsers import JSONParser
from .models import *
from .serializers import *

@csrf_exempt
def gene_detail(request, pk):
    """
    Retrieve, update or delete a code snippet.
    """
    try:
        gene = Gene.objects.get(pk=pk)
    except Gene.DoesNotExist:
        return HttpResponse(status=404)
```

```

if request.method == 'GET':
    serializer = GeneSerializer(gene)
    return JsonResponse(serializer.data)

```

urls.py

```

from . import api

urlpatterns = [
    path('api/gene/<int:pk>/', api.gene_detail),
]

```

4_207: Implementing POST

api.py

```

from rest_framework.decorators import api_view
from rest_framework.response import Response
from rest_framework import status

@api_view(['GET', 'POST'])
def gene_detail(request, pk):

    if request.method == 'POST':
        serializer = GeneSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data,
status=status.HTTP_201_CREATED)
        return Response(serializer.errors,
status=status.HTTP_400_BAD_REQUEST)

    try:
        gene = Gene.objects.get(pk=pk)
    except Gene.DoesNotExist:
        return HttpResponse(status=404)
    if request.method == 'GET':
        serializer = GeneSerializer(gene)
        return Response(serializer.data)

@api_view(['GET'])
def genes_list(request):
    if request.method == 'GET':
        gene = Gene.objects.all()
        serializer = GeneListSerializer(gene, many=True)
        return Response(serializer.data)

```

4_210: Class-based views in DRF

api.py

```

from rest_framework import generics
from rest_framework import mixins

```

urls.py

```
path('api/genes', views.GeneList.as_view()),
```

api.py

```
class GeneDetails(mixins.CreateModelMixin,
                  mixins.RetrieveModelMixin,
                  mixins.UpdateModelMixin,
                  mixins.DestroyModelMixin,
                  generics.GenericAPIView):
    queryset = Gene.objects.all()
    serializer_class = GeneSerializer

    def post(self, request, *args, **kwargs):
        return self.create(request, *args, **kwargs)

    def get(self, request, *args, **kwargs):
        return self.retrieve(request, *args, **kwargs)

    def put(self, request, *args, **kwargs):
        return self.update(request, *args, **kwargs)

    def delete(self, request, *args, **kwargs):
        return self.destroy(request, *args, **kwargs)
```